

保健統計学実習

第5日目

- 第13回 AI Deep learning演習(2) *Python Collaboratory使用
- 第14回 AI Deep learning演習(3) *Python Collaboratory使用
- 第15回 まとめ

滋賀医科大学NCD疫学研究センター
医療統計学部門

原田 亜紀子
(aharada@belle.shiga-med.ac.jp)

講義・演習スケジュール

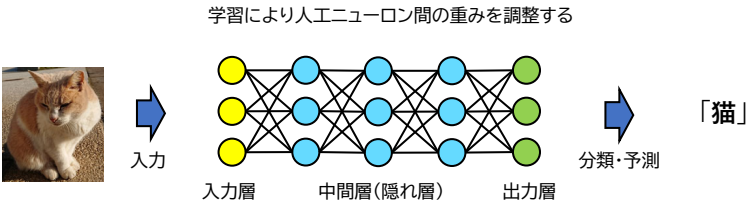
回	日	内容
1	8/27(木)	1 R, EZR の使い方, データセットの読み込み, 頻度集計
		2 記述統計, 相関
		3 エクセルの基礎, Python 設定, 基本コード
2	8/28(金)	4 仮説検定の基礎, カイ2乗検定, マクネマー検定
		5 t 検定, Wilcoxon 検定, サンプルサイズ設計
		6 調査データ解析
3	9/1(火)	7 分散分析
		8 重回帰分析
		9 ロジスティック回帰, Poisson 回帰
4	9/2(水)	10 生存時間解析
		11 クラスタ分析
		12 AI Deep learning 演習(1) *Python Collaboratory 使用
5	9/4(金)	13 AI Deep learning 演習(2) *Python Collaboratory 使用
		14 AI Deep learning 演習(3) *Python Collaboratory 使用
		15 解析実習まとめ(復習・課題の時間)

本日の講義内容

1. ニューラルネットワークによる多クラス分類
 - 損失関数、活性化関数
 - 深層学習用ライブラリTensorFlow、Keras
 - 手書き数字の画像の分類
 - 練習問題 Fashion item画像の分類
2. 畳み込みニューラルネットワークによるクラス分類
 - 食事分類(CNN)
3. YOLO(物体検出)
 - 食事画像から各種メニューの抽出
 - アノテーション体験

深層学習

深層学習では、人工ニューロンによるネットワークを何層にも重ね合わせ、対象の特徴を自動で抽出する。



層を深く積み重ねることで、複雑な入力データの分類や数値の予測を行える

深層学習の応用例

1. 画像認識

顔認証システム
不良品検知
画像検索
パン屋のレジサービス
医療の画像認識
自動運転
画像生成

2. 音声認識

音声アシスト
音声翻訳
議事録サービス

3. 自然言語処理

自動翻訳
chatbot
chatGPT

5

パーセプトロン・ニューラルネットワーク・ディープラーニング

パーセプトロン

データ → 重み1 → 重み2 → 重み3 → 誤差逆伝播法 → 出力
学習するたびに正解となるものへのむずびつきを強くし、不正解とのつながりを弱める

ニューラルネットワーク

データ → 入力層 → 中間層 → 出力層 → XX%
XX%

ディープラーニング

データ → 入力層 → 中間層(多層) → 出力層 → XX%
XX%

層を増やすと複雑な問題に対応
→ 勾配消失 → もっと層を増やす

6

原理

学習データ → 正解データ

入力データ → 予測計算
予測関数 → 出力データ

損失計算
損失関数 → 損失

パラメータ修正 → 勾配値 → 勾配計算

7

まずはうごかしてみよう

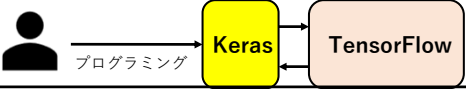
8

8

深層学習用のライブラリ

この講義では、TensorFlowとKerasというライブラリを用いて深層学習を学んでいく。

- TensorFlowはGoogleが開発している深層学習用のライブラリ。Meta(Facebook)が開発しているPyTorchとならんでよく使われるライブラリ。
- Kerasは深層学習用のライブラリをより簡単に使えるようにするためのインターフェイスを提供するGoogleが開発しているライブラリ。TensorFlowで直接プログラムを書くより簡単。TensorFlow以外のライブラリとも連携できる。



9

手書き文字の分類

Kerasにはサンプルデータセットとして、MNISTデータベース(Modified National Institute of Standards and Technology database)の手書きの0から9までの数字の画像が含まれている。
多層のニューラルネットワークを使って、これらの画像の分類を行う。



サンプルプログラムmnist.ipynbを実行していく。

10

忘れずに！！colaboratoryでのGPUの利用

時間制限はあるが、google colaboratoryの環境ではGPUを使って計算ができる。ディープラーニングでは、GPUを使うことで大幅に実行時間を短縮できる。



11

コードの説明

12

ニューラルネットワークと深層学習

今回作成したのはニューロンの層を二つ重ねただけのモデルだが、画像の分類という複雑な処理が可能となった。

深層学習では中間層の数を増やし、より複雑な問題を解けるようにしている。

Flatten層 中間層 出力層

x_1
 x_2
⋮
 x_{784}

ReLU関数

softmax関数

全結合の接続は省略

13

ライブラリとデータの読み込み

TensorFlowに付属したKerasを使う。

```
TensorFlowをtfという名称で読み込む
import tensorflow as tf
from tensorflow.keras import datasets

(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()
```

訓練用データ テスト用データ MNISTのデータを読み込む

14

原理

学習データ 正解データ

入力データ 予測計算 予測関数 出力データ

損失計算 損失関数

損失

勾配計算

勾配値

パラメータ修正

15

データのサイズと次元の確認

各次元の要素数を確認できる。

```
print("訓練用画像の枚数と画素数")
print(x_train.shape)

print("テスト用画像の枚数と画素数")
print(x_test.shape)
```

訓練用画像は60000枚 テスト用画像は10000枚

訓練用画像の枚数と画素数
(60000, 28, 28)

テスト用画像の枚数と画素数
(10000, 28, 28)

画像の画素数は28×28

16

画像を表示

訓練用画像の一部を表示してみる。

ax.imshow(x_train[i], cmap='Greys')

matplotlibのimshowを使うと2次元配列を画像として表示できる

グレースケール(白黒)で表示

正解のラベルがついている

17

カテゴリ変数のone-hotエンコーディング

機械学習では、本来数値ではない変数をプログラムで扱いやすくするために数値のラベルに置き換えて扱うことがある。
このような質的なデータについての変数をカテゴリ変数と呼ぶ。

例えば、血液型を次のように数値に置き換えて扱うことができる。

A型	→	0		A型	→	[1, 0, 0, 0]
B型	→	1		B型	→	[0, 1, 0, 0]
AB型	→	2		AB型	→	[0, 0, 1, 0]
O型	→	3		O型	→	[0, 0, 0, 1]

ベクトル形式で表現することもある

こちらの表現を**one-hotエンコーディング**といい、ディープラーニングでもよく用いられる。

18

画像ラベルのone-hotエンコーディング

手書き数字の画像には画像の種類を表す0から9までのラベルがつけられている。
これはカテゴリ変数で数値の大小に意味はない。
Kerasのto_categoricalを用いてone-hotエンコーディングを行う。

```
from tensorflow.keras.utils import to_categorical

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

元のラベルが保存された配列を渡す

全部で10クラスのラベルがある

19

モデルの作成

Kerasのsequential(直列)モデルを作成する。

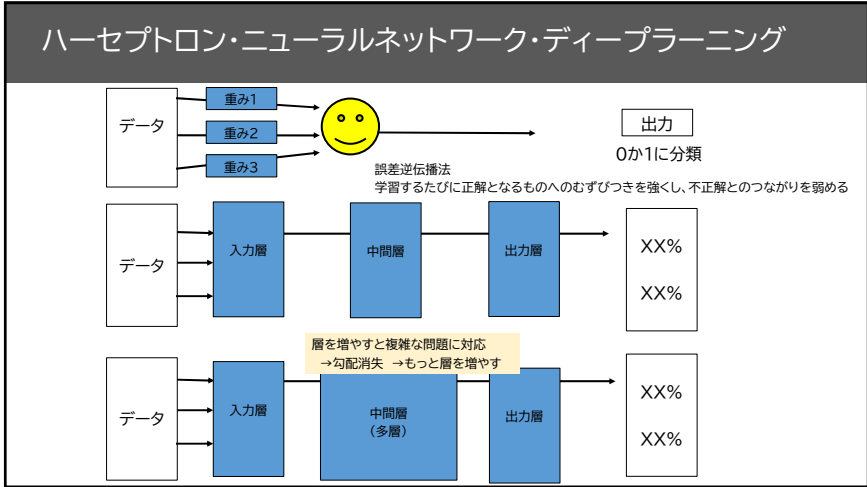
Sequentialをインポートしておく

```
from tensorflow.keras.models import Sequential

model = Sequential()
```

Sequentialでモデルを作成する。
この時点ではまだニューロンの層がない空っぽの状態。

20



21

Flatten層の追加

作成したモデルに**add**で層を順番に追加していく。
ここでは最初に画像を一次元配列に変換するFlatten層を追加する。
追加したい層をインポートしておく

```
from tensorflow.keras.layers import Dense, Dropout, Flatten  
model.add(Flatten(input_shape=(28, 28)))
```

addで追加する層を指定する

一番最初の層では入力データの形状を指定する必要がある。
(ここでは28画素×28画素の2次元配列)

22

一次元配列への変換

二次元配列として保存されている画素値のデータを一次元配列に変換してからニューラルネットワークに入力する。
(二次元配列のまま入力する方法もある。)

1	18	0
184	49	14
92	217	89

→ Flatten層で変換

1	18	0	184	49	14	92	217	89
---	----	---	-----	----	----	----	-----	----

23

中間層(隠れ層)の追加

中間層(隠れ層)として**dense**層を追加する。

Unitsで層の中のニューロンの数を指定する。
ここでは128個のニューロンを持つ層を追加する。

```
model.add(Dense(units=128, activation='relu'))
```

活性化関数としてReLU関数を使う

24

出力層の追加

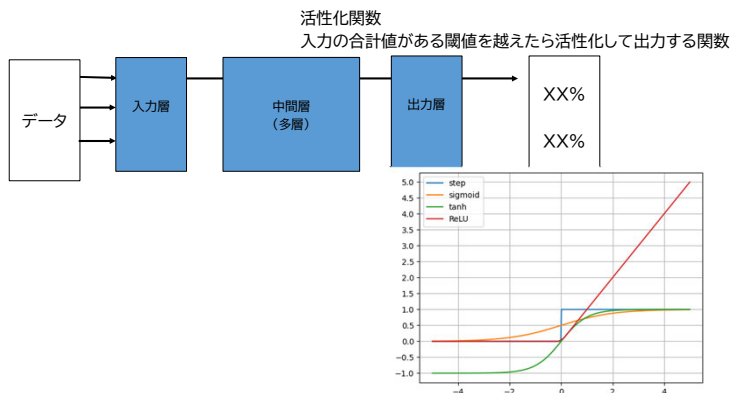
出力層として、分類するクラスと同じ数のニューロンを持つdense層を追加する。
Unitsで層の中のニューロンの数を指定する。
ここでは10個のニューロンを持つ層を追加する。

```
model.add(Dense(units=10, activation='softmax'))
```

活性化関数としてsoftmax関数を使う

25

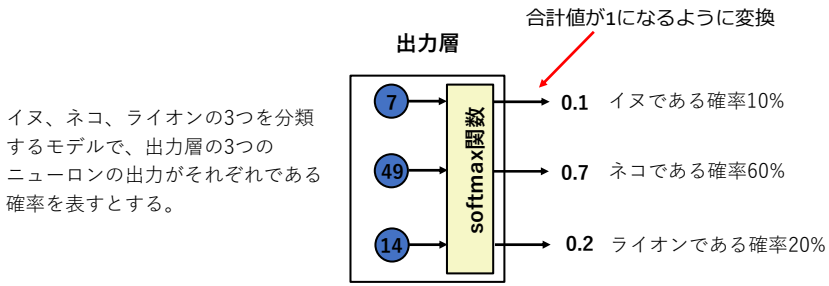
ニューラルネットワーク・ディープラーニング 活性化関数



26

Softmax関数

多クラス(2クラス以上)の分類では、活性化関数としてsoftmax関数を用いる。
softmax関数はすべての出力値の合計が1になるように、出力値を調整する。



27

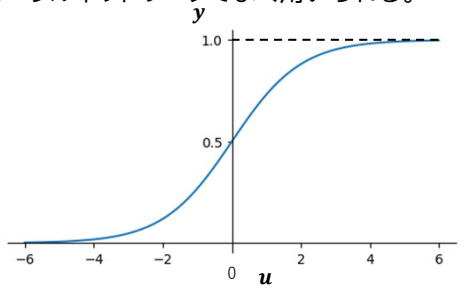
シグモイド関数

シグモイド関数は $x = 0$ で不連続に値が変化するのはではなく、 $y = 0$ から $y = 1$ までの範囲を連続的に値が変化する。
微分可能であるためニューラルネットワークでよく用いられる。

シグモイド関数

$$y = f(u) = \frac{1}{1 + e^{-u}}$$

$e = 2.718 \dots$ は自然対数の底



28

ReLU関数

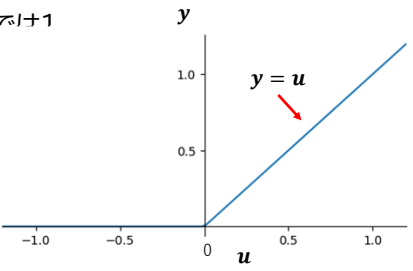
ReLU(Rectified Linear Unit)関数は入力値が負なら0、正なら入力値をそのまま出力する。
微分の値は $u < 0$ では0、 $u \geq 0$ では1

ReLU関数

$$y = f(u) = \begin{cases} u, & u \geq 0 \\ 0, & u < 0 \end{cases}$$

微分値の絶対値がシグモイド関数より大きい。

損失関数の勾配が小さくなりすぎず、ニューラルネットワークの層が深い場合に適している。



29

作成したニューラルネットワークの構成

入力画像 Flatten層 中間層 出力層 出力

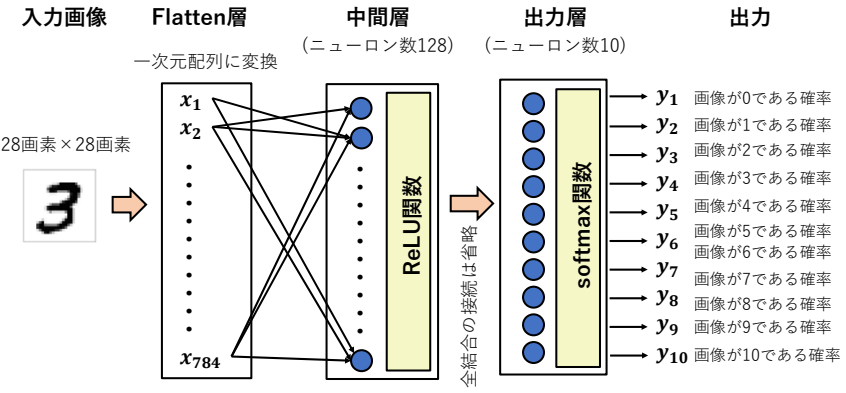
一次元配列に変換 (ニューロン数128) (ニューロン数10)

28画素 × 28画素

全結合の接続は省略

softmax関数

y_1 画像が0である確率
 y_2 画像が1である確率
 y_3 画像が2である確率
 y_4 画像が3である確率
 y_5 画像が4である確率
 y_6 画像が5である確率
 y_7 画像が6である確率
 y_8 画像が7である確率
 y_9 画像が8である確率
 y_{10} 画像が9である確率



30

モデルのコンパイル

層を追加したモデルで学習ができるようにするためには、モデルをコンパイルする必要がある。

損失関数を最小にするパラメータを探すアルゴリズムの指定（後述）

```
model.compile(optimizer='adam',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

損失関数として交差エントロピー誤差を用いる

学習結果の評価に使う指標を指定する。ここでは正解率を用いる。

31

モデルの概要

`model.summary()`
summaryでモデル全体の概要を確認する。

各層の出力の数 各層のパラメータ数

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

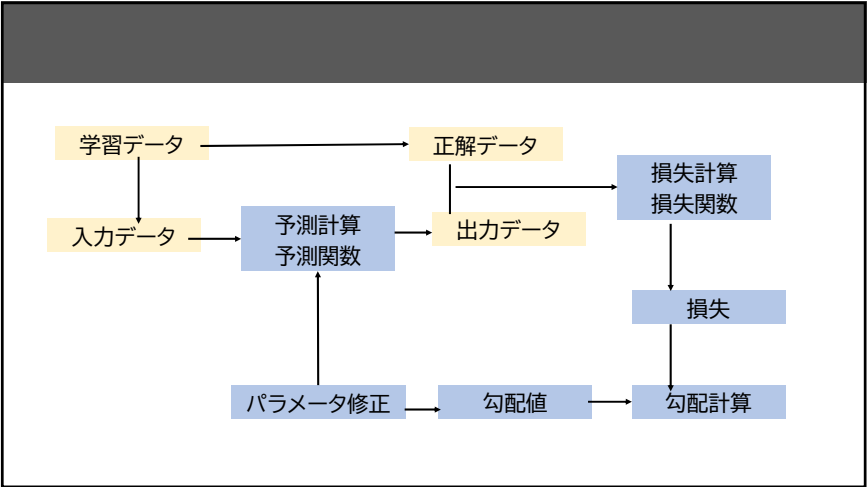
Total params: 101,770 (397.54 KB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)

784 × 128 + 128

入力値に対する重み バイアス

モデル全体で101770個ものパラメータがある

32



33

損失関数

ニューラルネットワークを使った分類モデルや回帰モデルを作るためには、正しい予測が行えるように入力値に対する重みとバイアスのパラメータを調整しなければならない。

モデルの予測とデータの実際の値とのずれを表す損失関数を導入し、損失関数の値が最小になるようにパラメータを最適化する。

34

パーセプトロン・ニューラルネットワーク・ディープラーニング

誤差逆伝播法
学習するたびに正解となるものへのむずびつきを強くし、不正解とのつながりを弱める

どれだけ間違えていたか → 損失関数

傾き右肩下がり 重みを大きくする 重みを小さくする 誤差が一番小さい

35

Adam

深層学習では扱うパラメーター数が膨大になるため、効率的に損失関数を最小にするパラメータを探す最適化アルゴリズムが必要。

最急降下法では学習率は固定されている。学習率が小さすぎると最適解に近づくのに時間がかかり、大きすぎると発散して最適解にたどり着けない。

損失関数の微分を使ってパラメータの値を更新

$$w_{i+1} = w_i - \eta \nabla L_i(w)$$

Adamでは、以前のステップの勾配の情報も使い、学習率を(更新の幅)を動的に変えながら効率的に最適解に近づく。

36

学習の実行

訓練データを使って学習するには**fit**を用いる。

epoch数は訓練データ全体を何回使って学習を行うかを表す。
ここでは訓練データ全体を10回使ってパラメータを調整する。

訓練用のデータ

```
epochs = 10
```

```
results = model.fit(x_train, y_train, epochs=epochs, validation_data=(x_test, y_test))
```

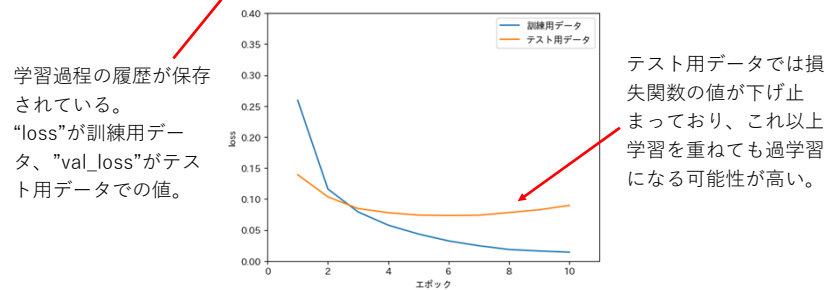
損失関数の値の変化等、学習過程の履歴が保存される。

テスト用のデータを渡しておけば、
テスト用のデータでも損失関数の
値や成果率を計算してくれる。

37

学習履歴の表示(損失関数の値の変化)

```
ax.plot( epo, results.history["loss"], label="訓練用データ")
ax.plot( epo, results.history["val_loss"], label="テスト用データ")
```

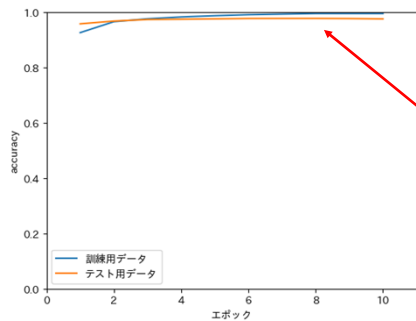


ネットワークの重みの初期値はランダムに設定されるため、学習のたびに結果の数値に多少の違いがでる。

38

学習履歴の表示(正解率の値の変化)

```
ax.plot( epo, results.history["accuracy"], label="訓練用データ")
ax.plot( epo, results.history["val_accuracy"], label="テスト用データ")
```



訓練用データでもテスト用データでも高い正解率を得られている。

39

学習済みモデルを用いた予測

predictに入力データを渡せば、学習済みのモデルで予測ができる。

テスト用データを使って画像の種類を表すラベルを予測させる。

```
y_pred = model.predict(x_test)
```

予測結果は、各分類ラベルが正解である確率の配列として求められる。

予測結果の例

[1.2e-12 9.5e-15 3.1e-11 1.0e-06 3.3e-18 9.8e-13 6.6e-17 9.9e-01 5.7e-13 4.1e-11]

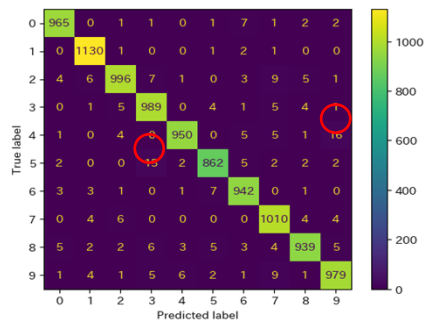
ラベルが0 ラベルが1 ラベルが2
である確率 である確率 である確率

ラベルが7である確率
(確率が最も大きい)

40

結果:混同行列の表示

混同行列を表示させれば、どの種類の画像の組み合わせが間違えやすいかなども確認できる。



9と4、3と5などの組み合わせが誤分類が多い。

41

練習問題 ファッションアイテム画像の分類

手書き数字の分類のサンプルプログラムを参考にし、ファッションアイテム(シャツ、ドレス、かばん、靴、等)の画像を分類するプログラムを実行しなさい。

ファイル『fashion_kadai.ipynb』



42

正解ラベルと画像の対応

ラベル値	画像に映っているもの
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

43

44

本日の講義内容

- 1. ニューラルネットワークによる多クラス分類
 - 損失関数、活性化関数
 - 深層学習用ライブラリTensorFlow、Keras
 - 手書き数字の画像の分類
 - 練習問題 Fashion item画像の分類
- 2. 畳み込みニューラルネットワークによるクラス分類
 - **食事分類(CNN)**
- 3. YOLO(物体検出)
 - 食事画像から各種メニューの抽出
 - アノテーション体験

数字分類→食品の写真

- 出典:電気通信大学 柳井研究室 **UEC FOOD 100 データセット**
Yuji Matsuda and Keiji Yanai: Recognition of Multiple Food Images by Detecting Candidate Regions, Proc. of IEEE International Conference on Multimedia and Expo (ICME), (2012)
100種類の食品の写真。各食品の写真には、食品の位置を示すバウンディングボックスが付与してあるデータベース。
→演習用に、12品目、主食・主菜・副菜・汁物から選んだ

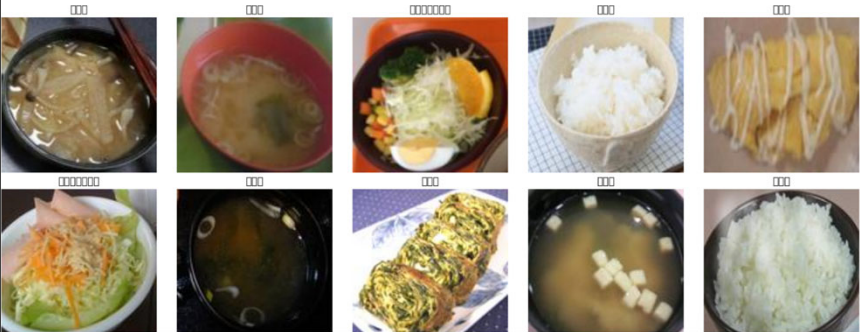
食品データ(左) 12品目(右)

	file_name	label	label_name
0	36_15803_0.jpg	1	味噌汁
1	36_15154_0.jpg	1	味噌汁
2	87_14990_0.jpg	10	グリーンサラダ
3	1_31_0.jpg	0	ごはん
4	67_14711_0.jpg	7	卵焼き
...	...	...	...
2336	36_13840_0.jpg	1	味噌汁
2337	36_14764_0.jpg	1	味噌汁
2338	1_5073_0.jpg	0	ごはん
2339	1_14834_0.jpg	0	ごはん
2340	87_13829_0.jpg	10	グリーンサラダ

2341 rows x 3 columns

	label	label_name
0	0	ごはん
1	1	味噌汁
2	2	鮭の塩焼
3	3	鶏の唐揚げ
4	4	豚カツ
5	5	ハンバーグ
6	6	豚肉の生姜焼き
7	7	卵焼き
8	8	納豆
9	9	冷奴
10	10	グリーンサラダ
11	11	豚汁

写真例



畳み込みニューラルネットワーク

畳み込みニューラルネットワーク(Convolutional Neural Network)は、**畳み込み層と最大値プーリング層で画像の2次元な特徴を抽出**し、それらを反映させた学習や予測を行うニューラルネットワーク。

「畳み込み層 + 最大値プーリング層」の組み合わせで特徴量マップを抽出。

特徴量マップの情報を利用して分類・予測

手前の畳み込み層は複数でも可

画像のまま入力

畳み込み層

最大値プーリング層

畳み込み層

最大値プーリング層

Flatten層

中間層

出力層

予測結果

49

colaboratoryでのGPUの利用

時間制限はあるが、google colabの環境ではGPUを使って計算ができる。ディープラーニングでは、GPUを使うことで大幅に実行時間を短縮できる。

ランタイム

ランタイムの種類を変更

ランタイムの種類

Python 3

ハードウェアアクセラレータ

☒ CPU ☒ T4 GPU ☐ A100 GPU ☐ L4 GPU

☐ TPU v2-8

プレミアム GPU を利用するには [コンピューティングコノートを追加購入](#)

キャンセル 保存

50

food_CNN_classification_学生版.ipynb

ファイル 編集 表示 挿入 ランタイム ツール ヘルプ

Q コマンド + コード + テキスト ▶ すべてのセルを実行 ▶ ドライブにコピー

Food Image Classification (UEC FOOD 100 subset)

主食・主菜・副菜・汁物の12品目を対象に、CNNで食品画像の分類を行います。MNIST・銀河画像分類の演習と同じ流れ(データ読み込み → モデル構築 → 学習 → 評価)で進めます。

事前に prepare_uecfood_classification_data.py を実行し、images/, train.csv, test.csv, class_map.csv を Google Drive にアップロードしておいてください。

1. データの読み込み

```
!pip install gdown -q

# ↓ 訓練用Google Driveに置いた classification_data.zip のファイルIDに書き換えてください
FILE_ID = "1PnoUvSRFXCE6tYmD_2HGvYi-Ck-sRx4LO"

!gdown --id {FILE_ID} -O classification_data.zip
!rm -rf classification_data
!unzip -q -o classification_data.zip -d .
```

51

CSVファイルの読み込み

pandasのデータフレームとして読み込む。

```
import pandas as pd

dir_path = 'classification_data/'

df_train = pd.read_csv(dir_path + 'train.csv')
df_test = pd.read_csv(dir_path + 'test.csv')
df_class_map = pd.read_csv(dir_path + 'class_map.csv')
```

訓練用データの CSV ファイル

テスト用データの CSV ファイル

52

CSVファイルの内容の確認

データフレームとして読み込んだ訓練用データのCSVファイルを表示する。`df_train`
(テスト用データも同じ形式で情報が保存されている)

訓練用画像のファイル名

全部で2341 枚の訓練用画像が含まれている

	file_name	label	label_name
0	36_15803_0.jpg	1	味噌汁
1	36_15154_0.jpg	1	味噌汁
2	87_14990_0.jpg	10	グリーンサラダ
3	1_31_0.jpg	0	ごはん
4	67_14711_0.jpg	7	卵焼き
...	...	...	...
2336	36_13840_0.jpg	1	味噌汁
2337	36_14764_0.jpg	1	味噌汁
2338	1_5073_0.jpg	0	ごはん
2339	1_14834_0.jpg	0	ごはん
2340	87_13829_0.jpg	10	グリーンサラダ
2341 rows x 3 columns			

各画像の正解ラベル

53

画像データの読み込み

```
from keras.preprocessing.image import load_img, img_to_array
import numpy as np

def load_images(df):
    images = []
    labels = []
    for _, row in df.iterrows():
        fname = dir_path + 'images/' + row['file_name']
        img = load_img(fname)
        img = img_to_array(img)
        images.append(img)
        labels.append(row['label'])
    return np.array(images), np.array(labels)

x_train, y_train = load_images(df_train)
x_test, y_test = load_images(df_test)
```

Kerasにはファイルから画像を読み込むための命令として **load_img**、画像を配列に変換する命令として **img_to_array** が用意されている。
リストされている画像ファイルを、for文で順番に読み込んでいく。
深層学習の入力データとして使えるように、画像データを多次元配列に変換する。

54

データのサイズと次元の確認

numpyの多次元配列はshapeで各次元の要素数を確認できる。

```
print("訓練用画像の枚数、画素数、チャンネル数")
print(x_train.shape)
print("テスト用画像の枚数、画素数、チャンネル数")
print(x_test.shape)
```

訓練用画像は 2341枚

テスト用画像は 585枚

訓練用画像の枚数、画素数、チャンネル数

(2341, 128, 128, 3)

テスト用画像の枚数、画素数、チャンネル数

(585, 128, 128, 3)

画像は画素数128×128、チャンネル数3(R,G,B)のカラー画像

55

画像を表示

```
import matplotlib.pyplot as plt

class_names = dict(zip(df_class_map['label'], df_class_map['label_name']))

fig, axes = plt.subplots(2, 5, figsize=(15, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(x_train[i].astype('uint8'))
    ax.set_title(class_names[y_train[i]])
    ax.axis('off')
plt.tight_layout()
plt.show()

matplotlibのimshowを使うと配列を画像として表示できる
```



56

ラベルのone-hotエンコーディング

```
from tensorflow.keras.utils import to_categorical
```

```
num_classes = len(df_class_map)
```

```
y_train_cat = to_categorical(y_train, num_classes)
```

```
y_test_cat = to_categorical(y_test, num_classes)
```

57

モデルの構築(CNN)

Kerasのsequentialモデルを作成する。

Sequentialをインポートしておく

```
from tensorflow.keras.models import Sequential
```

```
model = Sequential()
```

Sequentialでモデルを作成する。

この時点ではまだニューロンの層がない空っぽの状態。

58

畳み込み層の追加 (1)

作成したモデルに**add**で層を順番に追加していく。

最初に画像の畳み込み(コンボリューション)を行う**Conv2D**層を追加する。

畳み込みに用いる**フィルター**

(カーネル)の数を指定。

ここでは32個とした

カーネルの大きさの指定。

ここでは3画素×3画素とした。

```
from tensorflow.keras.layers import Conv2D
```

```
# 畳み込み層(1)
```

```
model.add(Conv2D(filters=32, kernel_size=(3, 3),  
activation='relu', input_shape=(128, 128, 3)))
```

活性化関数としてReLU関数を使う

一番最初の層では入力データの形状を指定する。

(128画素×128画素、3チャンネル(R,G,B))

一次元配列に変換せずに入力している

59

畳み込み層の追加 (2)

今回作成するモデルでは、特徴量マップに対してさらに畳み込みを行う。

畳み込みに用いる**フィルター(カーネル)の数**

を指定。

この層では64個とした

```
# 畳み込み層(2)
```

```
model.add(Conv2D(filters=64, kernel_size=(3, 3),  
activation='relu'))
```

他のパラメータは1層目の畳み込み層と同じにしている。

60

最大値プーリング層の追加

畳み込み層の後ろに最大値プーリングを行う**MaxPooling2D**層を追加する。

事前にインポートしておく

```
from tensorflow.keras.layers import MaxPooling2D
model.add(MaxPooling2D(pool_size=(2, 2)))
```

プーリングを行う小領域の大きさの指定。
ここでは2画素×2画素の幅で最大値プーリングを行う。

61

二組目の畳み込み層とプーリング層の追加

今回作成するモデルでは、ここまで追加した畳み込み層と最大値プーリング層と同じ構成の層をもう一組追加する。

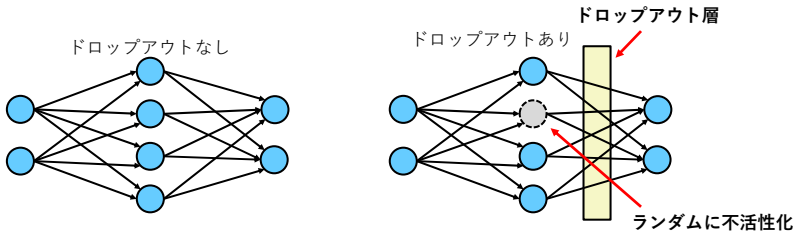
```
model.add(Conv2D(filters=32, kernel_size=(3, 3),
                  activation='relu'))
model.add(Conv2D(filters=64, kernel_size=(3, 3),
                  activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
```

62

ドロップアウト

ニューラルネットワークが訓練用データに対して最適化されすぎると過学習の状態になり、訓練用データ以外に対する予測精度が低下してしまう。

ドロップアウトは深層学習における**過学習を抑制する手法**のひとつで、ニューロンからの出力を一定の割合でランダムに不活性化(出力を無視)する。



63

ドロップアウト層の追加

ドロップアウトを行う**Dropout**層を追加する

事前にインポートしておく

```
from tensorflow.keras.layers import Dropout
model.add(Dropout(0.25))
```

不活性化するニューロンの割合を指定する。
ここでは25%とした。
0.2~0.5程度の範囲で設定

64

後半部分の層の追加

畳み込み層と最大値プーリング層から得られた特徴量マップから分類予測を行うための層を追加していく。

特徴量マップを一次元配列に変換する

```
model.add(Flatten())  
model.add(Dense(units=128, activation='relu'))  
model.add(Dropout(0.25))
```

出力層の前にもドロップアウト層を追加

特徴量マップの情報で分類するための中間層

65

出力層の追加

二つのクラスを分類するための出力層として、ニューロンを一つだけのdense層を追加する。

```
# 出力層: 12クラス分類なのでsoftmax  
model.add(Dense(units=num_classes, activation='softmax'))
```

12クラス分類なので、softmax関数を活性化関数として使う

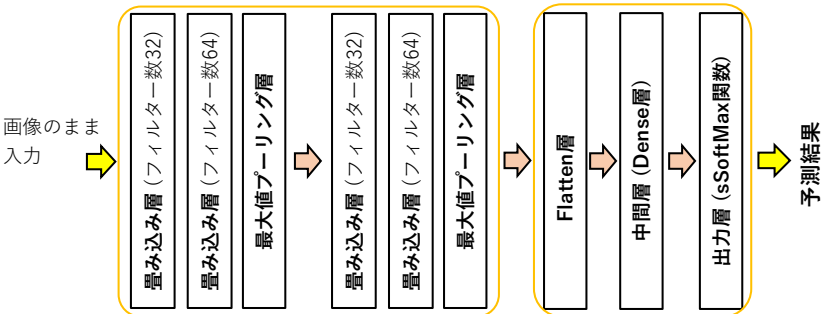
66

構築したニューラルネットワークの構成

今回構築したニューラルネットワークの構成は下図のようになる。

特徴量マップを抽出

特徴量マップの情報を利用して分類



67

モデルのコンパイル

層を追加したモデルで学習ができるようにするためには、モデルをコンパイルする必要がある。

損失関数を最小にするパラメータを探すアルゴリズムの指定

```
model.compile(optimizer='adam',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

損失関数として交差エントロピー誤差を用いる。

多クラス(3クラス以上)の場合は categorical_crossentropyを指定する。

学習結果の評価に使う指標を指定する。
ここでは正解率を用いる。

68

学習の実行

訓練データを使って学習するにはfitを用いる。

epoch数は訓練データ全体を何回使って学習を行うかを表す。
ここでは訓練データ全体を10回使ってパラメータを調整する。

訓練用のデータ

```
epochs = 10  
  
results = model.fit(x_train, y_train, epochs=epochs, validation_data=(x_test, y_test))
```

損失関数の値の変化等、学習過程の履歴が保存される。

テスト用のデータを渡しておけば、テスト用のデータでも損失関数の値や成果率を計算してくれる。

69

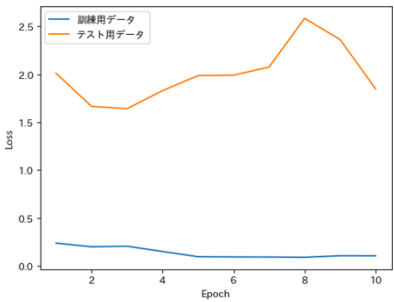
```
epo = range(1, epochs + 1)
```

```
fig, ax = plt.subplots()  
ax.plot(epo, results.history["loss"], label="訓練用データ")  
ax.plot(epo, results.history["val_loss"], label="テスト用データ")  
ax.set_xlabel("Epoch")  
ax.set_ylabel("Loss")  
ax.legend()  
plt.show()
```

70

70

学習履歴の表示(損失関数の値の変化)



このモデルは典型的な過学習を示します(訓練Accuracy~97%、検証Accuracy~60%で頭打ち、
訓練Lossはほぼ0まで下がるのに検証Lossはepoch3以降増加)。

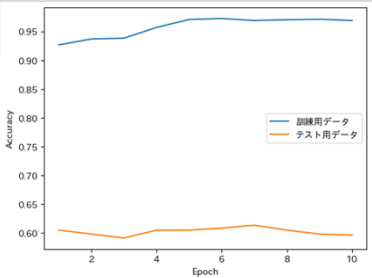
ネットワークの重みの初期値はランダムに設定されるため、学習のたびに結果の数値に多少の違いがある。(講義のスライドと全く同じにはならない)

71

学習履歴の表示(正解率の値の変化)

```
fig, ax = plt.subplots()  
ax.plot(epo, results.history["accuracy"], label="訓練用データ")  
ax.plot(epo, results.history["val_accuracy"], label="テスト用データ")  
ax.set_xlabel("Epoch")  
ax.set_ylabel("Accuracy")  
ax.legend()  
plt.show()
```

このモデルは典型的な過学習を示します(訓練Accuracy~97%、検証Accuracy~60%で頭打ち、
訓練Lossはほぼ0まで下がるのに検証Lossはepoch3以降増加)。



72

学習済みモデルを用いた予測

predictに入力データを渡せば、学習済みのモデルで予測ができる。
テスト用データを使って画像の種類を表すラベルを予測させる。

```
y_pred = model.predict(x_test)
y_pred_label = np.argmax(y_pred, axis=1)
```

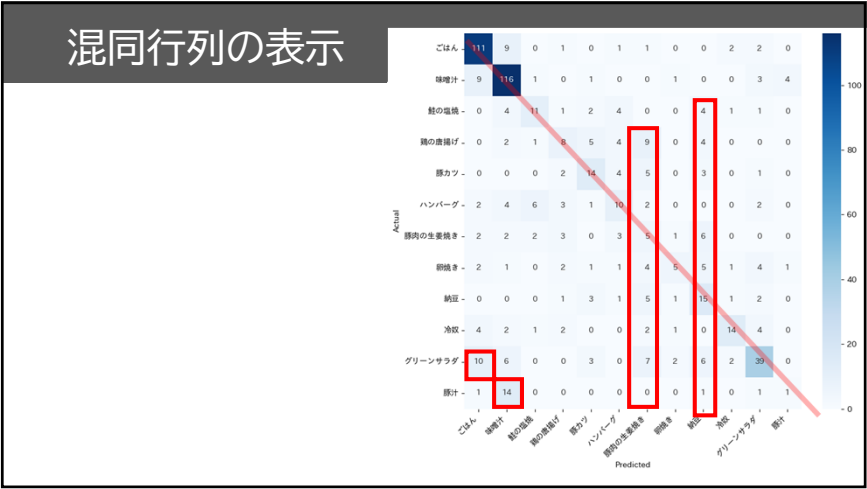
```
# テスト画像といくつかの予測結果を表示
fig, axes = plt.subplots(2, 5, figsize=(15, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(x_test[i].astype('uint8'))
    correct = "○" if y_pred_label[i] == y_test[i] else "×"
    ax.set_title(f"{correct} 予測: {class_names[y_pred_label[i]]} 正解: {class_names[y_test[i]]}", fontsize=9)
    ax.axis('off')
plt.tight_layout()
plt.show()
```

テスト用画像と予測結果の一部を表示

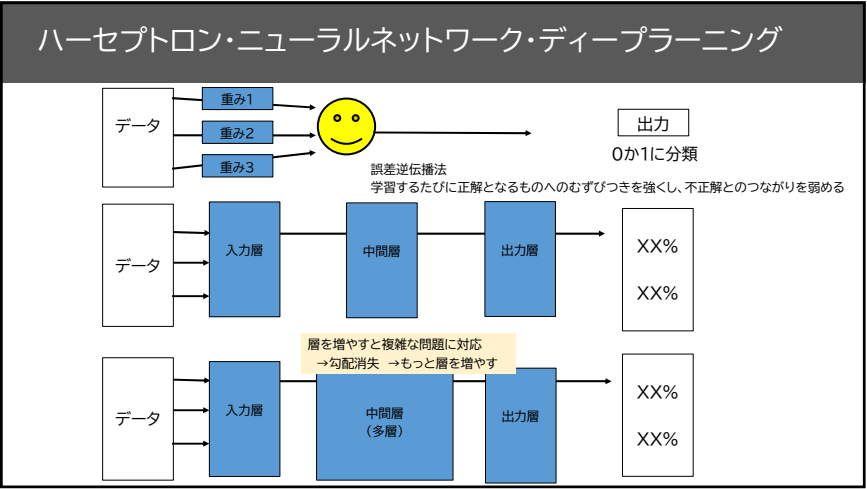
テスト用画像の一部とモデルの予測結果を表示してみる。
ここに示したものはほとんど正しく予測できている。



Model: "sequential"		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
conv2d_1 (Conv2D)	(None, 124, 124, 64)	18,496
max_pooling2d (MaxPooling2D)	(None, 62, 62, 64)	0
conv2d_2 (Conv2D)	(None, 60, 60, 32)	18,464
conv2d_3 (Conv2D)	(None, 58, 58, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 29, 29, 64)	0
flatten (Flatten)	(None, 53824)	0
dense (Dense)	(None, 128)	6,889,600
dropout (Dropout)	(None, 128)	0
flatten_1 (Flatten)	(None, 128)	0
dense_1 (Dense)	(None, 128)	16,512
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 12)	1,548
Total params: 6,964,012 (26.57 MB)		
Trainable params: 6,964,012 (26.57 MB)		
Non-trainable params: 0 (0.00 B)		



77



78

モデルの修正 food.CNN_classification_自己学習版.ipynb

- 今のCNNのまま、学習のやり方を改善する(データ拡張・EarlyStopping・クラス不均衡対策)
- 事前学習済みモデル(転移学習)に土台を差し替える(一番効果が大きいはず)

実行前に予想してみましょう:
さきほどの混同行列では、味噌汁と豚汁が頻繁に混同されていました。データ拡張・class weight・転移学習を加えると、この混同は直ると思いますか? それとも、データ量を増やしても解決しない種類の混同だと思いますか? 実行後、自分の予想と結果を比べてみてください。

79

12. 精度改善を試してみよう

- ここまでで「実行→解釈」は一通り終わりました。ここからは、実際に精度を上げられるか試す時間です。効果が大きい順に、2つのやり方を用意しています。両方試して、混同行列がどう変わるか比べてみましょう。
- 12-1: 今のCNNのまま、学習のやり方を改善する(データ拡張・EarlyStopping・クラス不均衡対策)
- 12-2: 事前学習済みモデル(転移学習)に土台を差し替える(一番効果が大きいはず)

80

12-1: 今のCNNのまま、学習のやり方を改善

- データ拡張: 回転・反転・拡大でデータのバリエーションを増やす
- EarlyStopping: val_lossの改善が止まったら自動で学習を打ち切る
- class_weight: 件数の少ないクラス(生姜焼きなど)を軽視しないようにする

81

81

12-2: 転移学習(事前学習済みのモデル)に差し替える

- ImageNetで事前学習済みのMobileNetV2を土台にする

82

82

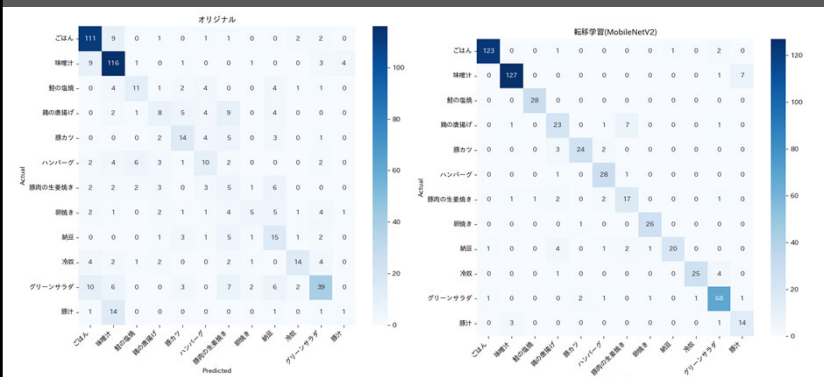
前回 → 12-1



83

83

前回 → 12-2



84

84



85

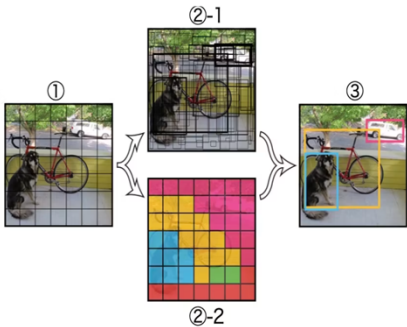
85



86

86

物体検出 YOLO



① マス目に分解

②-1 バウンディングボックスを配置

信頼度が0に近いほど、バウンディングボックスの中身は「背景」を表しています。
・信頼度が1に近いほど、バウンディングボックスの中身は「物体」を表しています。

②-2 何を表しているか区分

③ ②より判断

87

以下のファイルを使用する

- 先ほどの100種類のデータを使用して、あらかじめ学習してアルモデルを用いる
- 学習済みモデルを使用した演習を行う

<使用データ>

- 「 food_YOLO_exercise.ipynb 」
- 「best.pt 」
- 検証データファイル

88

88

food_YOLO_exercise.ipynb

food_YOLO_exercise.ipynb
ファイル 編集 表示 挿入 ランタイム ツール ヘルプ

Q コマンド + コード + テキスト | ▶ すべてのセルを実行 | ドライブにコピー

Food Detection Exercise (学生用)

学習済みのYOLOモデルを使って、配布された写真から複数の食品を検出してみましょう。学習は済んでいるので、このノートブックではモデルを読み込んで、推論だけです。

+ コード + テキスト

1. モデルと写真の準備

```
!pip install gdown -q
# ↓ 読解用Google Driveに置いた food_yolo_exercise.zip のファイルIDに書き換えてください
FILE_ID = "15190FPyBrsuXincRBJ11Y04EUL-0kS"
!gdown --id (FILE_ID) -O food_yolo_exercise.zip
!unzip -q -o food_yolo_exercise.zip -d .
```

2. 学習済みモデルの読み込み

best.pt (先生が事前に用意した、12品目・学習済みの重み)を読み込みます。

89

下記階層にファイルを保存

```
from ultralytics import YOLO

# 環境に合わせて書き換えてください
model_path = '/content/drive/MyDrive/Colab Notebooks/food_yolo/best.pt'
photo_dir = '/content/drive/MyDrive/Colab Notebooks/food_yolo/test_photos/'

model = YOLO(model_path)
```

best.py : 学習済みモデル

test_photosのフォルダ内に、検証ファイルを保存

90

```
# 配布された写真の1枚を試してみる(ファイル名は実際のものに書き換えてください)
#result = model.predict(photo_dir + "karaage2.png", conf=0.01, verbose=False)[0]
result = detect_and_show_yolo(photo_dir + "karaage2.png", confidence=0.02)
```

```
result =
detect_and_show_y
olo(photo_dir +
"karaage2.png",
confidence=0.02)
```

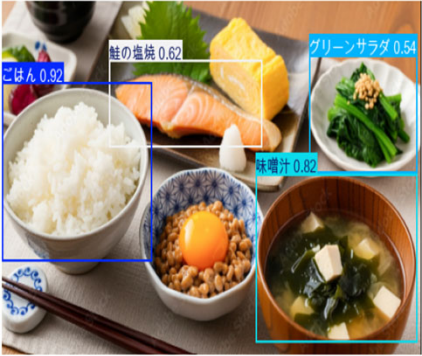
赤字部分を判定したい
ファイルに変更して実行
する

ごはん: confidence = 0.92 味噌汁: confidence = 0.83 グリーンサラダ: confidence = 0.68 鶏の唐揚げ: confidence = 0.23 グリーンサラダ: confidence = 0.05 グリーンサラダ: confidence = 0.05 ごはん: confidence = 0.02 鶏の唐揚げ: confidence = 0.02 グリーンサラダ: confidence = 0.02 グリーンサラダ: confidence = 0.01

91

ごはん: confidence = 0.87
豚汁: confidence = 0.83
納豆: confidence = 0.53
グリーンサラダ: confidence = 0.06

92



ごはん: confidence = 0.92
味噌汁: confidence = 0.82
鮭の塩焼: confidence = 0.62
グリーンサラダ: confidence = 0.54

93



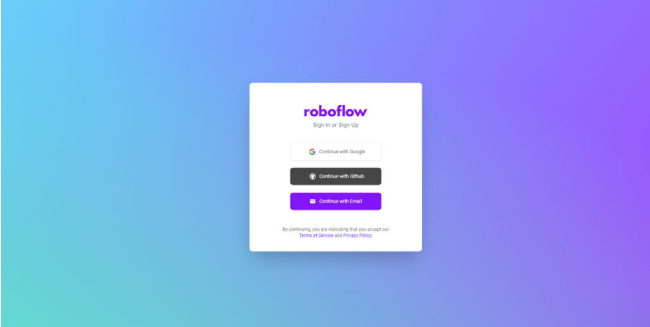
味噌汁: confidence = 0.61
ごはん: confidence = 0.06
味噌汁: confidence = 0.01
ごはん: confidence = 0.01

94

自分でアノテーションから
経験してみる

95

Roboflowを使用



96

Robo flowを使用

1. アカウント作成
Gmailか自身の大学メールアカウントかで作成
「Choose your plan:」の項目に、「Public Plan」

2. ログイン後「プロジェクト」作成

roboflow

Workspace

Research Plan • 1 Member

Agent

Projects

Asset Library

Workflows

Models

Search projects

Sort: Date Edited

New Folder

New Project

Object Detection

NEW_an

Edited 3 minutes ago

35 Im... • 1 M...

Object Detection

定食メニュー...

Edited 6 days ago

29 Im... • 3 M...

97

プロジェクト作成

roboflow

myfirstworkspace

Open Team • 1 Member

Projects

Workflows

Monitoring

Deployments

Notifications

Settings

Universe

There are no projects in this workspace.

Create a project and upload images to start labeling, training, ai

your computer vision model.

New Project

View a Tutorial

98

モデル	区分	説明	例
Object Detection	物体検出	画像内の対象物の種類と位置を認識する	画像内から犬を検出
Single-Label Classification	シングルラベル画像分類	対象の画像に一つのラベルを付け識別する	良品写真 or 不良品写真
Multi-Label Classification	マルチラベル画像分類	対象の画像に複数のラベルを付け識別する	画像に犬と猫が写っていることを識別
Instance Segmentation	インスタンスセグメンテーション	画像内に映っている対象物を種類毎に個別に認識し領域を抽出する	自動運転などで車・人を個体毎に識別
Keypoint Detection	キーポイント検出	画像内に映っている対象物の向きを識別する	骨格検知、顔の向きの検知
Multimodal	マルチモーダル	画像を表現する区分・内容を定義し画像からテキストを生成する	画像からテキストを生成し、画像の説明等を行う

99

Let's create your project.

Project Name

Visibility

Licenses

Annotation Group

Object Detection

Identify objects and their positions with bounding boxes.

Classification

Instance Segmentation

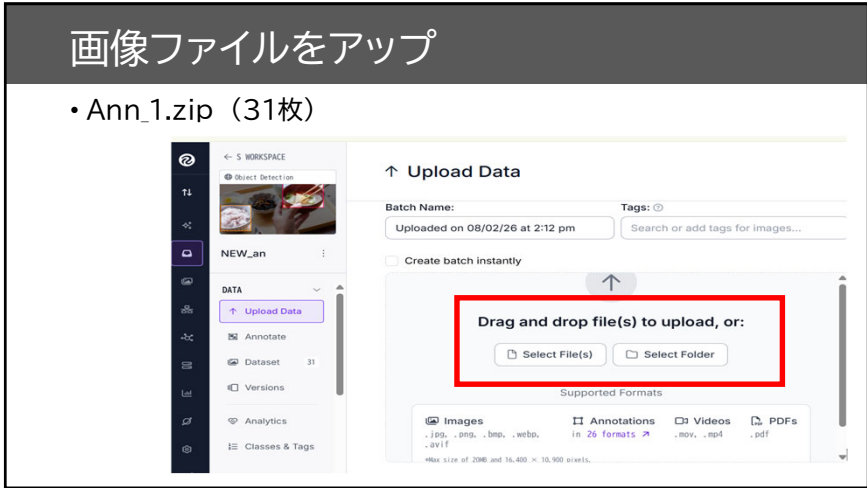
Keypoint Detection

Multimodal

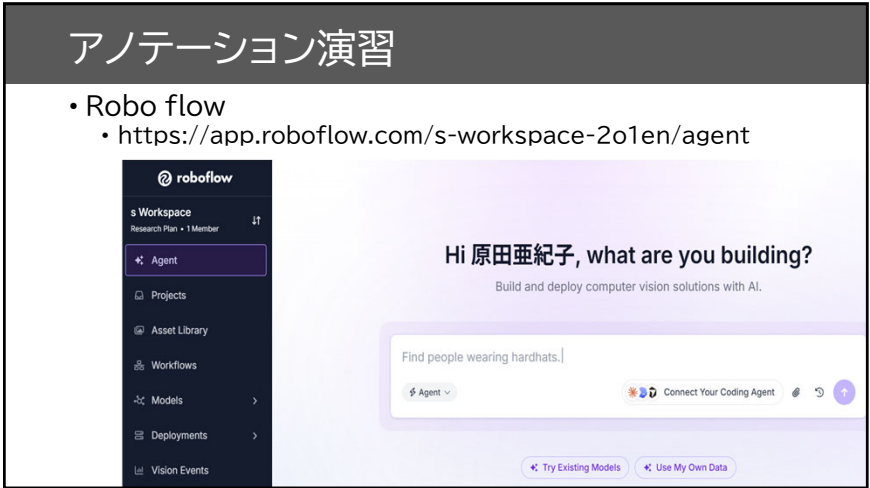
Semantic Segmentation

Continue With Public

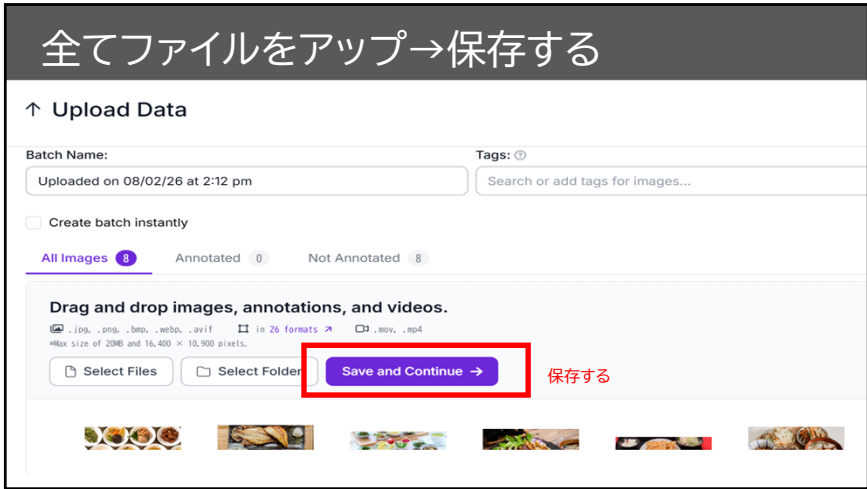
100



101



102



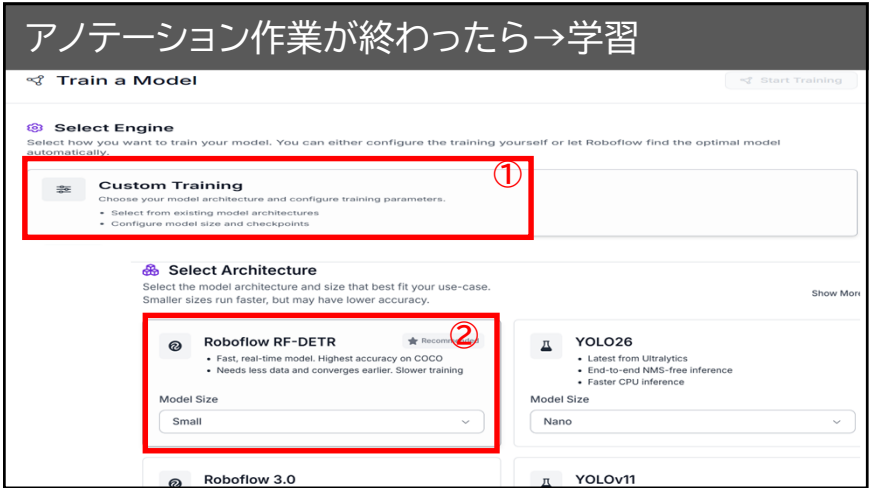
103



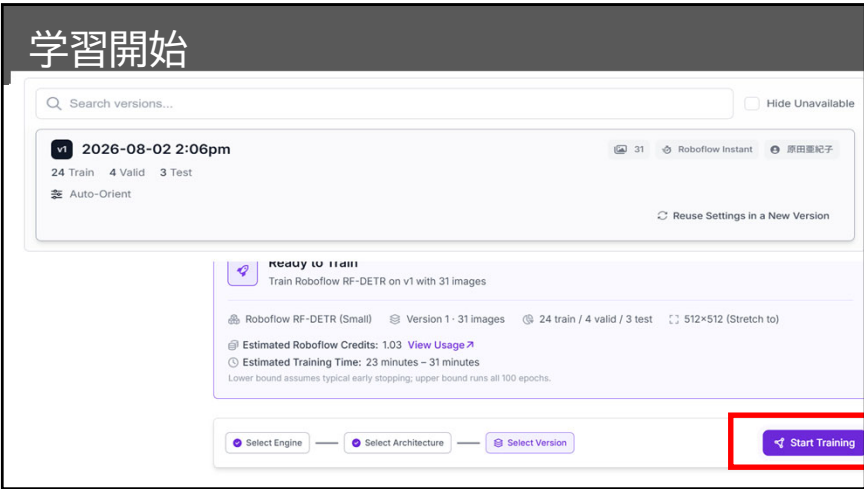
104



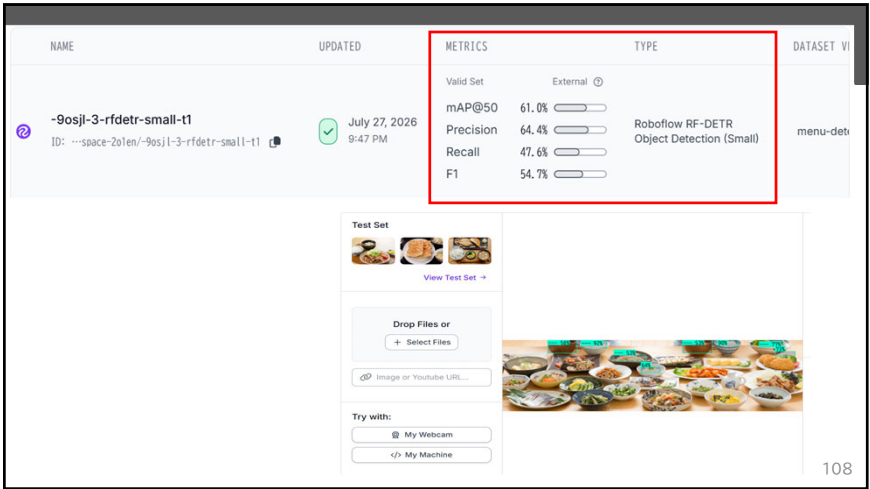
105



106

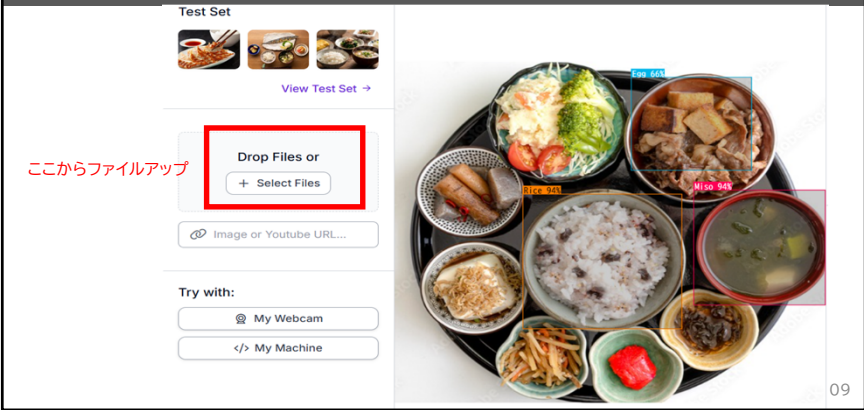


107



108

検証データ(Ann2-4.zip)で判定してみる



109

検証



110

アノテーション→学習が間に合わない場合

- 既存の学習モデルで試してみる (学習済みモデル.txt)

[\[new_an-ch7ze-1-rfdetr-small-t1\]モデル](#)

- <https://app.roboflow.com/s-workspace-2o1en/new-an-ch7ze/models/s-workspace-2o1en/new-an-ch7ze-1-rfdetr-small-t1>

[\[-9osjl-3-rfdetr-small-t1\]モデル](#)

- <https://app.roboflow.com/s-workspace-2o1en/-9osjl/models/s-workspace-2o1en/-9osjl-3-rfdetr-small-t1>

111

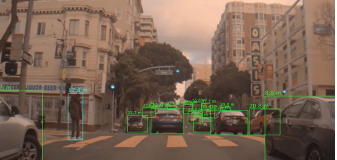
111

畳み込みニューラルネットワークの応用

今回の講義ではそれぞれの画像に何が写っているかだけで分類したが、1枚の画像(動画も含む)の中のどこに何が写っているかを検出する物体検知、特定のものが写っている領域を画素単位で分割する(塗分ける)セマンティックセグメンテーション等、応用範囲は広い。

物体検知

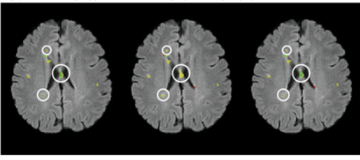
車の自動運転での利用



Nvidia社ブログ <https://blogs.nvidia.co.jp/blog/> より

セマンティックセグメンテーション

脳のMRI画像からの病変の検出



T. Brosch et al. 2016より

112



113



114

実習補足

- 演習課題の整理
- パッケージ
 - JMPのライセンス更新
 - Python
 - SAS
- 教科書

115

本実習の課題

下記の1、2、3について取り組んで提出してもらい、これらを出席と合わせて評価に用います。

1. 各演習日の課題：以下の課題を設定しています(4回分設定)。

①解析パッケージR・EZRを用いた解析実施報告

第1日目 血圧区分、箱ひげ図の作成

第2日目 t検定演習(トウモロコシの例)

第3日目 ~~compareの実施~~ **ロジスティックまたはPoisson演習課題**

第5日目 因子分析因子得点を用いたクラスター分析、市町村人気度の比較

②Googleフォームを用いた調査 調査票の回答:回答する

調査票の作成:リンク先を送付

2. 実習で実施した調査票の作成を通じ、解析を行う視点から、調査票作成時に注意すべき点を考察する。(全員)

例:調査項目の設定、コーディングなど

3. 卒業研究を行うまでに、経験、修得しておいたほうがよいこと **(全員)**

卒業研究でデータ解析を行うことを想定し、統計学、データ解析において、今後必要な知識、トレーニング(本演習の復習を含め)などをあげる。

116

参考資料(ワークブック系)
*資料として保存してあります

• 下川敏雄.EZR による医学統計入門 第2.0版 (和歌山県立医科大学附属病院 臨床研究センター)

EZR による医学統計入門

第2.0版

下川敏雄

和歌山県立医科大学附属病院 臨床研究センター

2.0.1 データの読込み・変換・保存

2.0.2 データの加工

2.0.3 データの分析・グラフの作成

2.0.4 データの出力

117

EZRの基本操作、解析の流れ

1. データの読み込み

2. データの整備(変数ビューの設定)

3. データの加工

4. 分析・グラフの作成

5. 分析結果の出力



118

EZRの画面

ファイル 編集 アクティビタセット 統計解析 グラフと表 ツール ヘルプ 標準メニュー

データセット: Dataset 変換 保存 モデル: エクスポートモデルなし

統計解析

Dataset <- readSPSS("F:/health.sav", rownames=FALSE, stringsAsFactors=TRUE, tolower=FALSE)
editDataset(Dataset)

結果出力

各操作(データセット・解析・グラフ)メニュー

選択したメニューのコードが表示。コードで分析も可能(学習用に使える)

119

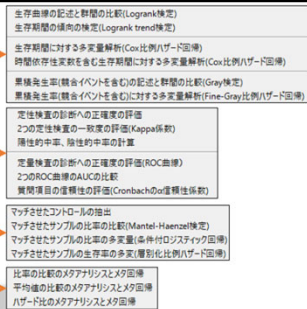
EZRにおけるメニュー
【統計解析】

名義変数の解析
連続変数の解析
カテゴリカル変数の解析
生存率解析
検査の正確性の評価
マッドドック解析
メタアナリシスとメタ分析
必要サンプルサイズの計算

検定分布
比率の信頼区間の計算
1標本の比率の検定
2群の比率の差の信頼区間の計算
2群の比率の比の信頼区間の計算
分散分析の導入と検定
分散分析の検定と検定比率の検定(Fisherの正確検定)
対応のある比率の検定(二群分散分析の検定, McNemar検定)
対応のある2群以上の比率の検定(Cochran Q検定)
比率の傾向の検定(Cochran-Armitage検定)
二値変数に対する多変量解析(ロジスティック回帰)
連続変数の検定
外れ値の検定と除外(Snimov-Grubbs検定)
正規性の検定(Kolmogorov-Smirnov検定)
平均値の信頼区間の計算
1標本の平均値の検定
2群の平均値の検定(t検定)
2群間の平均値の検定(t検定)
対応のある2群間の平均値の検定(paired t検定)
3群以上の分散性の検定(Bartlett検定)
3群以上の分散の平均値の検定(一元配置分散分析(one-way ANOVA))
対応のある2群以上の平均値の検定(反復測定分散分析)
複数の因子で平均値の検定(多変量分散分析(multi-way ANOVA))
連続変数で検定した2群以上の平均値の検定(共分散分析(COVA))
相関係数の検定(Pearsonの相関係数検定)
相関係数の検定(Spearmanの順位相関係数)
2群間の検定(Mann-Whitney U検定)
対応のある2群間の検定(Wilcoxon符号付順位相関係数)
3群以上の検定(Kruskal-Wallis検定)
対応のある3群以上の検定(Friedman検定)
連続変数の傾向の検定(Jonckheere-Terpstra検定)
相関係数の検定(Spearmanの順位相関係数)

120

[統計解析]



121

[グラフ]



122

1. 血圧区分(変数のカテゴリー化)、箱ひげ図の作成ができる
→血圧区分ごとの食塩摂取量について、箱ひげ図を作成

2. 血圧(SBP,DBP)、年齢、食塩摂取について、散布図、相関を検討し、4変数の関係性について、気づいた点を記述せよ。

適宜、必要なグラフを使用し説明すること

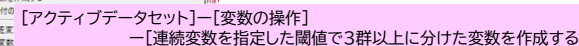
ヒント:

血圧同士の相関は???

食塩摂取と血圧の相関は???

年齢と収縮期血圧、拡張期血圧の散布図、相関は???

123



124

収縮期血圧区分

連続変数を1つ選択

age
dbp
HTN_c
id
salt
sbp_cc

新しい変数名: **sbp_cc**

所値と各グループ名を入力してください

レベルグループ名 1:	120-129	閾値 1:	130
レベルグループ名 2:	130-139	閾値 2:	140
レベルグループ名 3:	140-149	閾値 3:	150
レベルグループ名 4:	150-159	閾値 4:	160
レベルグループ名 5:	160-169	閾値 5:	170
レベルグループ名 6:	170-	閾値 6:	

閾値
☒ ≤ (以上)
☐ > (より大きい)

ヘルプ OK キャンセル

> #新しい変数 sbp_cc を作成しました。

```
table(health$sbp_cc, exclude=NULL)
```

120-129	130-139	140-149	150-159	160-169	170-
267	90	41	14	6	3

> #####連続変数を指定した閾値で9群以上に分けた変数を作成する#####

125

収縮期血圧区分と食塩摂取

統計解析

連続変数の解析

2群間の平均値の比較(t検定)

126

第2日目課題(2) 次回まで

- 簡単な(web)調査票を作成しますので、10問程度からなる調査案を考えておいてください
 - 対象: 大学生を対象にした調査(例:喫煙、ストレス、睡眠など)
 - 調査計画:
 - 調査期間
 - どのような調査項目(既存の調査票を入れる)
 - 変数表
 - 分析(集計案)
- 次回: Google formを使用したWeb調査の予定

127

t 検定の実施

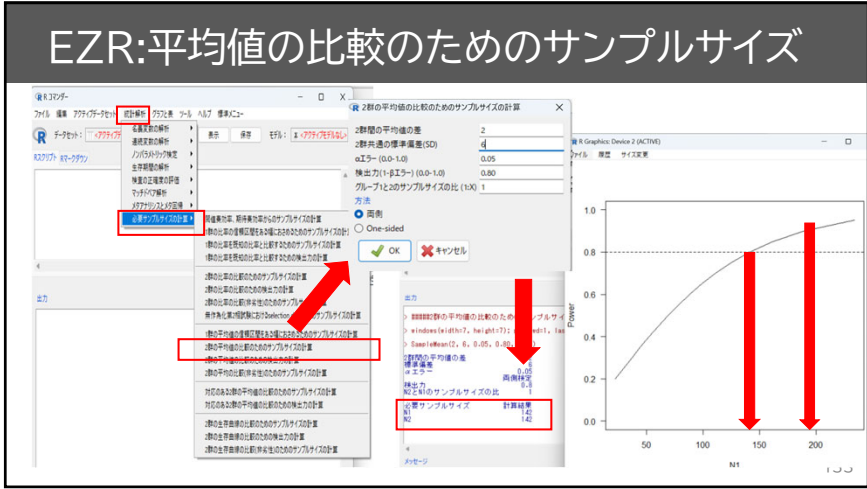
統計解析

連続変数の解析

2群間の平均値の比較(t検定)

[統計解析]—[連続変数の解析]—[2群間の平均値の比較(t検定)]

128



第3日目

1. Logistic 脳動脈瘤 例題

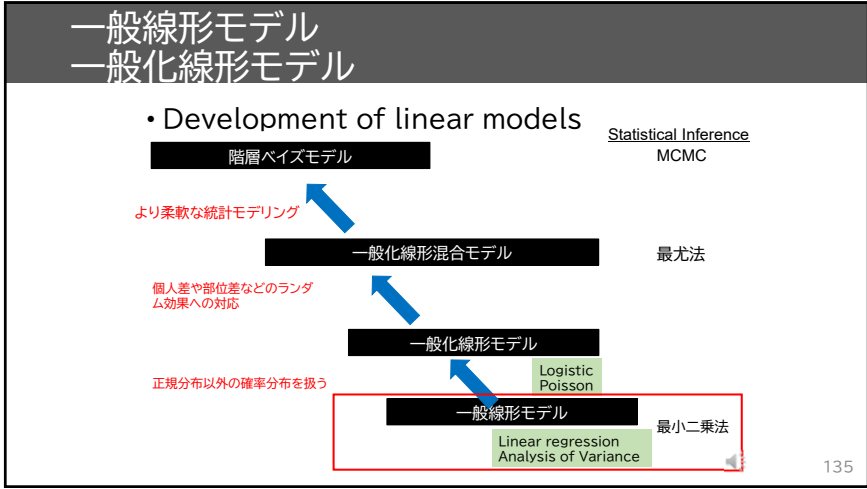
2. Poisson NEMS1988 例題

3. 調査票: 各自の調査票の作成

- 調査票を作成
- リンク先を入力
- コード表の作成

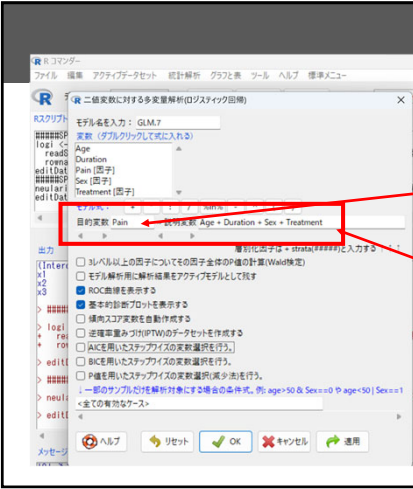
いずれか提出(追加課題)

134



ロジスティック回帰

136



- 式を作成

Pain(あり・なし)

年齢、期間、治療

137

Poisson distribution

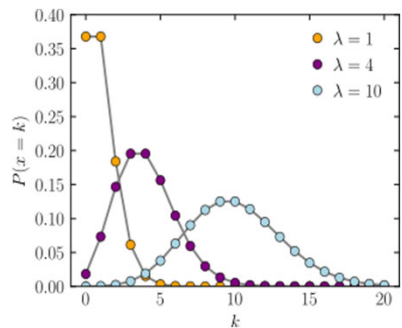
- 歪んでいる
- 非負の値のみをとる
- **分散は平均に等しい**

$$f(k; \lambda) = \Pr(X=k) = \frac{\lambda^k e^{-\lambda}}{k!},$$

where

- k is the number of occurrences ($k=0,1,2, \dots$)
- λ is equal to the expected value $E(x)$ of X and also to its variance $V(E)$.

$$\lambda = E(X) = \text{Var}(X).$$



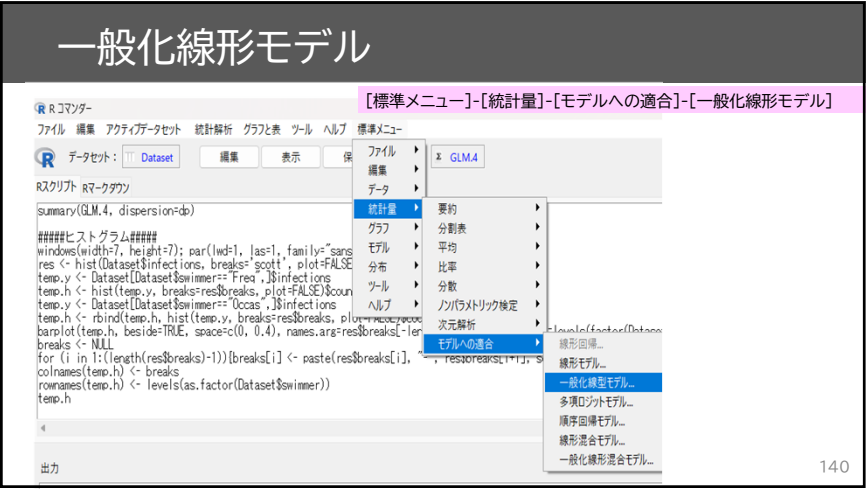
138

過分散の対応

1. モデルの誤差の調整
 - Dispersion parameter
$$\hat{\phi} = \frac{\text{Pearson } \chi^2}{\text{Residual degrees of freedom}}$$
$$= \frac{\sum_i (y_i - \hat{\mu}_i)^2 / \hat{\mu}_i}{\text{Residual degrees of freedom}}$$
2. Poisson分布よりばらつきの大きな負の二項分布を用いる

139

一般化線形モデル



140

一様化線形モデル

モデル名を入力: GLM5

変数 (ダブルクリックして式に入れる)

age (因子)

gender (因子)

infections

location (因子)

swimmer (因子)

モデル式

Operators (click to formula):

スプライン/多項式:

スプラインの自由度: 5

Indices or names of row(s) to remove

部分集合の表現

Weights

リンク関数版 (ダブルクリックで選択)

リンク関数

from属変数:infections

独立変数:age gender location swimmer

infections ~ age + gender + location + swimmer

Poisson回帰

分布:Poisson分布

リンク関数:log

141

過分散調整

Call:

glm(formula = infections ~ age + gender + location + swimmer,

family = poisson(log), data = Dataset)

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-0.12261	0.25197	-0.487	0.62653
age[T.20-24]	-0.37442	0.23599	-1.587	0.11260
age[T.25-29]	-0.18973	0.23917	-0.793	0.42762
gender[T.Male]	-0.08985	0.20648	-0.435	0.66345
location[T.NonBeach]	0.53454	0.19613	2.725	0.00642 **
swimmer[T.Occas]	0.61149	0.19304	3.168	0.00154 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 3.379853)

Null deviance: 824.51 on 286 degrees of freedom

Residual deviance: 755.43 on 281 degrees of freedom

AIC: 1139.8

142

過分散の対応

1. モデルの誤差の調整

Dispersion parameter

$$\hat{\phi} = \frac{Pearson\chi^2}{Residual\ degrees\ of\ freedom}$$
$$= \frac{\sum_i (y_i - \hat{\mu}_i)^2 / \hat{\mu}_i}{Residual\ degrees\ of\ freedom}$$

2. Poisson分布よりばらつきの大きな負の二項分布を用いる

143

負の二項分布

##negative binominal

install.packages("MASS")

library(MASS)

result_nb<-glm.nb(infections~age + gender + location

+ swimmer, data=Dataset)

summary(result_nb)

exp(coef(result_nb)) # Exponentiated coefficients

144

負の二項

```
Call:
glm.nb(formula = infections ~ age + gender + location + swimmer,
data = Dataset, init.theta = 0.5759778332, link = log)

Coefficients:
(Intercept)      -0.06082    0.23914   -0.254   0.79924
age[T.20-24]     -0.42924    0.23433   -1.832   0.06898
age[T.25-29]     -0.26250    0.24050   -1.081   0.27506
gender[T.Male]    -0.14002    0.20789   -0.674   0.50063
location[T.NonBeach] 0.54554    0.19605    2.783   0.00539 **
swimmer[T.Occas]  0.60305    0.18965    3.180   0.00147 **

Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Negative Binomial(0.576) family taken to be 1)

Null deviance: 289.90  on 286  degrees of freedom
Residual deviance: 269.13  on 281  degrees of freedom
AIC: 904.69

> exp(coef(result_nb)) # Exponentiated coefficients
(Intercept)      age[T.20-24]    age[T.25-29]    gender[T.Male] location[T.NonBeach]
0.9409927      0.6510050      0.7691229      0.8693440      1.7255466
swimmer[T.Occas]
1.8276786
```

水泳頻度と場所が有意
リスク比

たまにvs.頻回 1.83
海以外vs.海 1.73

145

145

AIC

- Poisson AIC: 1139.8
- 負の二項分布 AIC: 904.69
- ゼロ過剰Poisson AIC: 966.2640
- ゼロ過剰負の二項分布 AIC: 907.8795

146

146

R code ゼロ過剰Poisson, NB

```
## Zero-inflated Poisson
install.packages("pscl")
library(pscl)

result_zi <- zeroinfl(infections~age + gender + location + swimmer, data=Dataset)
summary(result_zi)

## Zero-inflated negative binomial
result_zinb <- zeroinfl(infections~age + gender + location + swimmer,
data=Dataset,dist="negbin")
summary(result_zinb)

## AIC
c(AIC.model1=AIC(GLM.1), AIC.model2=AIC(result_nb),AIC.model3=AIC(result_zi),
AIC.model4=AIC(result_zinb))
```

147

147

課題3-1

脳動脈瘤保有による破裂のリスクを評価したい

- N=1027のデータ(data=rapture)

• 下記変数

- Rupture:動脈瘤破裂の有無
- Male:男性
- Age:年齢
- Number aneurysm:動脈瘤の個数
- Hypertension:高血圧
- Hyperlipidemia:脂質異常
- fh.SAH:SAHの家族歴
- Bifurcation:分岐部に瘤あり
- Bleb:嚢胞上の構造あり
- MCA:中大脳動脈
- VABA:椎骨基底動脈瘤
- Size:大きさ

• Logistic回帰を用い、動脈瘤破裂のリスク因子の特定、破裂予測モデルを構築する

- ROC曲線、予測確率プロットなどを示せ

148

148

課題3-2

- 1987 年と 1988 年に実施された米国医療費調査 (NMES) データ (data=nmes)。
 - メディケアの対象となっている 66 歳以上の個人データ
 - 4,406 名 19変数
- 下記変数を使用し、医療機関受診回数との関連を検討する
 - 従属変数 Visits: 医療機関訪問回数
 - 独立変数
 - Hospital 入院期間
 - Health 主観的健康 "poor", "average" (reference category), "excellent"
 - Chronic 慢性疾患の有無
 - Adl ADL "limited" or not "normal"
 - Age Age in years (divided by 10) 10 歳階級
 - Gender (M, F)
 - Married 結婚しているか (Y, N)
 - School 教育年数
 - Income 世帯収入 (USD 10,000)
 - Insurance private insurance に加入しているか (Y, N)

149

149

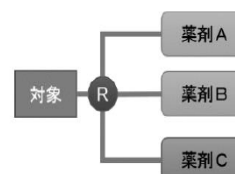
データの型： ①一元配置分散分析

One-way Analysis of variance (ANOVA)



Clinical Question

神経障害性疼痛患者がランダムに割り付けられ、3種類の除痛薬(A,B,C)のいずれかが投与された。



Data structure

● P : 被験者

藥劑A P P P P P P P P P P

藥劑B P P P P P P P P P

薬剤C P P P P P P P P P

A・B・Cの比較を行いたい
時点ごとの測定はない

150

150

データの型: ②繰り返し測定

Repeated measures ANOVA

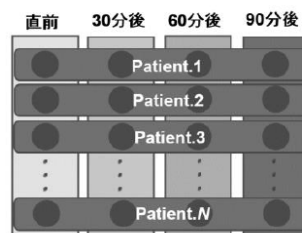


Clinical Question

N名の神経障害性疼痛患者に対して、新薬の除痛薬を投与し、投与直前、30分後、60分後、90分後のVASの変化を調査した。



Data structure



時点ごとの評価
薬は1種類(比較はない)

151

151

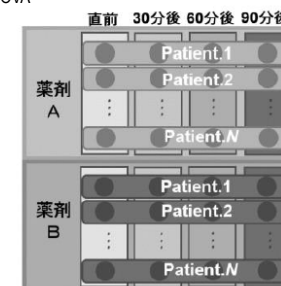
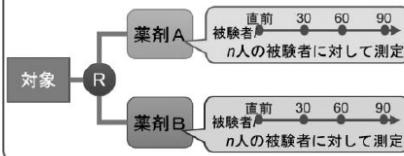
データの型: ③反復測定

Repeated measures ANOVA



Clinical Question

神経障害性疼痛患者がランダムに割り付けられ、3種類の除痛薬(A,B)のいずれかが投与され、投与直前、30分後、60分後、90分後のVASの変化を調査した。



時点ごとの評価
薬も比較

152

152

データの型： ④二元配置分散分析

Clinical Question

神経障害性疼痛患者がランダムに割り付けられ、3種類の除痛薬(A,B,C)のいずれかが投与された。この試験では、年齢(65歳以上、65歳未満)を割り調整因子とした。

対象

R

薬剤A

65歳未満 65歳以上

薬剤B

65歳未満 65歳以上

薬剤C

65歳未満 65歳以上

Data structure

薬の比較
対象者属性(年齢)の比較

65歳未満 65歳以上

薬剤A P P P P P P P P P P P P P P P P

薬剤B P P P P P P P P P P P P P P P P

薬剤C P P P P P P P P P P P P P P P P

P : 被験者

153

153

データセットの形式

• 対応あり(反復測定されている)かどうか
データセットの配列

分散分析・反復測定(対応あり)データ

dog	COL1	COL2	COL3	COL4	COL5
1	900	792	792	768	873
2	1510	1189	1109	1028	1277
3	1540	1082	1049	1148	1310
4	1082	885	902	885	1311
5	1180	885	754	754	951
6	1311	984	1000	984	1279

分散分析・対応なし・混合モデルデータ

time	inu	y
1	1	900
2	1	792
3	1	792
4	1	768
5	1	873
1	2	1510
2	2	1189
3	2	1109
4	2	1028
5	2	1277
1	3	1540
2	3	1082
3	3	1049
4	3	1148
5	3	1310
1	4	1082
2	4	885
3	4	902
4	4	885
5	4	1311
1	5	1180
2	5	885
3	5	754
4	5	754
5	5	951
1	6	1311
2	6	984
3	6	1000
4	6	984
5	6	1279

154

154

[アクティブデータセット]-[変数の操作]-[複数の変数を縦に積み重ねた・・・]

155

155

2) ①ログランク検定

$O_j = (d_{ij} - E(d_{ij}))$

生存時間

- 時間に関して層別解析
 - ある時点tにおける生存・死亡の2×2表を作成
 - at riskの合計人数が N_t で、そのうち D_t が死亡
- 帰無仮説: 2群で死亡率が同じ
 - 期待死亡数 $= D_t \times \frac{N_{tA}}{N_t}$
 - 群Aの期待死亡数
- 帰無仮説からのズレ $= D_{tA} - D_t \times \frac{N_{tA}}{N_t}$
 - 観察値-期待値
- 死亡時点ごとの結果
 - 各時点のズレのスコアを足し合わせる $U_A = \sum (D_{tA} - D_t \times \frac{N_{tA}}{N_t})$

ログランク検定
併合スコア U_A をその標準誤差で割った量

$$\frac{\sum (D_{tA} - D_t \times \frac{N_{tA}}{N_t})}{\sqrt{\sum \frac{N_{tA} N_{tB} D_t (N_t - D_t)}{N_t^2 (N_t - 1)}}$$

	死亡	生存	合計
群A	D_{tA}	$N_{tA} - D_{tA}$	N_{tA}
群B	D_{tB}	$N_{tB} - D_{tB}$	N_{tB}
	D_t	$N_t - D_t$	N_t

156

156

生存時間解析

gehan.csv
gehan.sav

R コマンド

ファイル 編集 アクティブデータセット 統計解析 グラフと表 ツール ヘルプ 標準メニュー

データセット: Dataset

R スクリプト R マークダウン

```
for (i in 1:length(ka$strat))
  for (j in 1:length(ka$strat))
    title(xlab = "Number at risk",
          ylab = "Legend", legend = "topright", legend.title = "Legend",
          res = NULL,
          res = NULL,
          ka.summary.table <- summary(
            ka.summary.table
```

出力

```
> (res <- survdiff(Surv(week, remiss==1)~drug, data=Dataset, rho=0, na.action = na.omit))
Call:
survdiff(formula = Surv(week, remiss == 1) ~ drug, data = Dataset,
na.action = na.omit, rho = 0)

      N Observed Expected (O-E)^2/E (O-E)^2/V
drug=6-WP      21         0      19.3         5.46
drug=CONTROL    21      10.7       8.77      18.8
```

157

157

生存時間解析

生存曲線の記述と群間の比較(Logrank検定)

観測期間の実数(1つ選択) イベント(1), 打ち切り(0)の実数(1つ選択)

drug remiss week drug remiss week

選択する変数はCtrlキーを押しながらクリック。

層別化変数(0~1つ選択)

drug remiss week drug remiss week

方法

☒ logrank

☐ Peto-Peto-Wilcoxon

異なる層の区別

☒ 色分け

☐ 線幅

☐ 線の太さ

凡例の書式

☒ 右上へ横並び

☐ 下部へ横並び

☐ マウス指定へ横並び

X軸の単位

☒ そのまま

☐ 日を月に交換

☐ 日を年に交換

☐ 月を年に交換

Post-hoc検定

(比較する群が2つの場合)

☒ No

☐ Bonferroni

☐ Holm

オプション

☒ 打ち切りをマークで表示する

☐ 95%信頼区間を表示する

☐ 異なる層毎に別の図に表示する

☒ At riskのサンプル数を表示する

☐ Y軸をパーセント表示にする

生存率を表示するポイント(観測期間の単位が日で1年生存率なら365) <none>

X軸の範囲(左端, 右端) 例: 0, 365 <auto>

Y軸の範囲(下端, 上端) 例: 0.0, 1.0 <auto>

X軸のラベル <auto>

Y軸のラベル <auto>

一部のサンプルが解析対象にしない場合の条件式。例: age>50 & Sex==0 & age<50 | Sex==1

<全ての有効なケース>

ヘルプ リセット OK キャンセル 適用

158

158

レポート課題: クラスター分析の例題

<B. Candyデータ>
階層クラスター、k-meansクラスターでの各クラスター別の平均値を比較してみる

R コマンド

ファイル 編集 アクティブデータセット 統計解析 グラフと表 ツール ヘルプ 標準メニュー

データセット: candy.csv

R スクリプト R マークダウン

```
Table
colMeans(fisher.test(.Table)) # 列の
res <- NULL
res <- fisher.test(.Table)
Fisher.summary.table <- tbl
colMeans(fisher.summary.table)
Fisher.summary.table
```

出力

```
> ##SPSSデータセットのインポート##
> readSPSS(
  filename="C:/Users/kyu/Desktop/20260904/レポート課題/Cluster/ClusterData/candy.csv",
  stringsAsFactors=TRUE, tolerance=1e-06)
```

「Candy.cluster.out.csv」に
・階層
・k-meansのクラスター情報
・チョコバーの属性
がまとめている。

階層、k-meansのクラスターごとの特徴を比較してみる

→ ANOVA
箱ひげ図

159

159

40